

REPLAYING WITH REALISTIC LATENT VECTORS IN GENERATIVE CONTINUAL LEARNING

Hyemin Jeong, Seong-Woong Kim, Dong-Wan Choi *

Department of Computer Science and Engineering, Inha University, South Korea
jhm2335@naver.com, wauri6@gmail.com, dchoi@inha.ac.kr

ABSTRACT

In generative continual learning for GANs, *replay-based* methods mitigate forgetting of past knowledge by retraining either synthesized images from the previous generator (*generative replay*) or real images from the memory buffer (*memory replay*) during training each new task. However, despite its strength at memory efficiency, generative replay often fails to produce realistic images, due to the gap between synthetic and real images especially in challenging datasets. Although memory replay can address this issue by storing real images, it still suffers from the practical limit of memory space, gradually deteriorating the quality of generated images. In this paper, we propose a novel *mixed replay* method, called *Realistic Latent Vector Fitting (RactoFit)*, which not only effectively resolves the drawbacks of generative and memory replay but also combine their strengths, memory utilization yet realism preservation. To efficiently utilize the memory space, our proposal is to maintain *realistic* latent vectors corresponding to previous real images, rather than storing those images themselves. In addition to replaying with these stored latent vectors, we also design a new optimization technique for latent vectors that can generate more realistic images from the previous generator, not just taking random vectors as in generative replay. Through our experimental analysis, we demonstrate that our method outperforms the baseline methods of continual learning for GANs in terms of the standard metrics for generative models, along with qualitative results clearly showing the highest level of realism of our generated images.

1 INTRODUCTION

With the success of huge generative models (Kang et al., 2023; Ramesh et al., 2022; Sauer et al., 2022; Rombach et al., 2022), generative AI is increasingly prevalent in different industries and aspects of our daily lives. Generative adversarial networks (GANs) have been one of the most popular generative models, introducing various architectures (Mirza & Osindero, 2014; Odena et al., 2017; Brock et al., 2019; Zhu et al., 2017; Karras et al., 2019) and diverse loss functions (Arjovsky et al., 2017; Gulrajani et al., 2017; Miyato et al., 2018). GANs’ popularity is due to their strengths at generating higher-quality samples than statistical models such as VAEs (Kingma & Welling, 2014) and requiring much less training cost than diffusion-based models (Ho et al., 2020). Hence, GANs are being used in many practical applications to provide high-quality images (Yi et al., 2018; Schlegel et al., 2017).

In order to achieve the practical usefulness, these GAN-based applications essentially need to expand their generative knowledge to be able to continuously produce new kinds of images. However, simply fine-tuning a GAN for a new task leads to *catastrophic forgetting* (McCloskey & Cohen, 1989; McClelland et al., 1995) to the point that it becomes no longer able to generate realistic images of previously learned tasks. As in the context of continual learning, it is not practical to assume that all previous samples are accessible during training for the current task, which makes it further challenging for GANs to retain the realistic quality of generated images throughout the continual learning steps.

In continual learning (CL), *replay-based* methods have been popularly used to preserve previous knowledge by re-training either synthesized images from the previous generator, called *generative replay* (Shin et al., 2017; Wu et al., 2018), or real images stored in the memory buffer, called *memory replay*, (Rios & Itti, 2019) during the training of current samples. With respect to generative CL for GANs, generative replay has more often adopted than memory replay, as GANs are typically employed for feeding synthetic images of old tasks to another model being continuously trained. However, to be shown in our experimental results of Figure 5, generative replay tends to gradually lose the GAN’s strength to produce realistic images for previous tasks, particularly when the underlying distribution of real data is highly complex. This is mainly because there surely exists a gap between synthetic images of previous tasks

* Corresponding Author

and true real images of each new task, but generative replay equally considers them ‘real’ and performs a joint training over two different types of samples. Consequently, the GAN would generate more and more unrealistic images over the continual steps, leading to a cumulative degradation of the GAN’s performance. On the other hand, memory replay can be better at preserving realism of generated images by replaying real images stored in the memory buffer. However, to continuously capture the data distribution of all previous tasks, it would be essential to store a large number of previous images, requiring a huge memory space.

In this paper, we propose a *mixed replay* approach, called *Realistic Latent Vector Fitting* (RactoFit), which can consistently preserve high-quality of data generation as in memory replay yet without having to store numerous old images as in generative replay over a given sequence of continual tasks. Our first strategy is about how to utilize the buffer space in a more efficient way. Instead of storing real images in the buffer, we suggest regeneration of samples by storing *realistic* latent vectors corresponding to real images of previous tasks. Since latent vectors are much smaller in size than raw images (e.g., 4% of each image size in CIFAR-10), we can regenerate a larger number of images corresponding to those vectors, and thereby effectively capturing the underlying data distribution. To obtain these realistic latent vectors, each corresponding to a real image of some specific task, we use GAN inversion techniques (Xia et al., 2023; Huh et al., 2020) with the trained GAN at the point.

In addition to replaying with realistic latent vectors in the memory buffer, we also perform generative replay using the previous generator to further enhance the knowledge preservation by additional synthetic samples. Unlike the conventional generative replay methods taking random latent vectors mostly from a Gaussian distribution, we devise a new optimization technique for latent vectors in order to obtain more realistic vectors than random ones, from which the generator can synthesize higher-quality images. More specifically, we start with random vectors that would generate some synthetic images, and then train those vectors themselves for their corresponding images to be optimized in terms of realism. Figure 1 highlights the effectiveness of our techniques using either inverted or optimized latent vectors so as to extract more realistic knowledge from a trained GAN. It is clearly observed from the histogram on predicted confidence as well as the representative samples, random vectors tend to yield quite low-quality images, whereas we can generate more realistic images from the latent vectors obtained by the two proposed techniques of RactoFit.

In our experiments, we show that our RactoFit method consistently achieves the best quantitative results in terms of the well-known evaluation metrics for GANs like FID and fitting capacity on various benchmark datasets. We also provide supportive qualitative results showing that RactoFit can generate the most realistic images across all continual learning tasks.

2 RELATED WORKS

There are three main groups of generative continual learning (GCL) approaches for GANs: regularization-based, expansion-based, and rehearsal-based methods.

Regularization-Based. Regularization-based methods maintain the previous knowledge by incorporating a regularization term into the loss function when learning a new task. Seff et al. (2017) apply EWC (Kirkpatrick et al.,

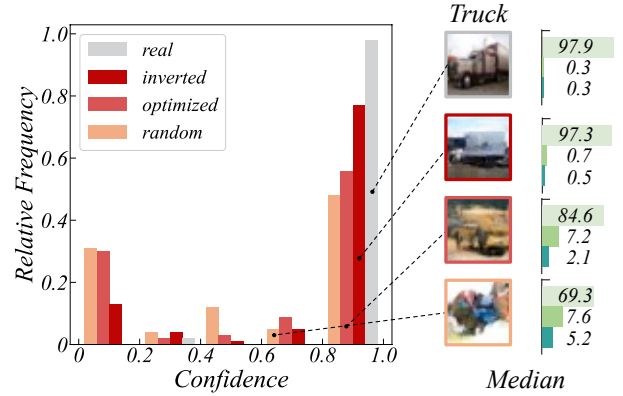


Figure 1: Comparison on latent vector quality for generating realistic ‘truck’ images from a well-trained GAN with CIFAR-10. Our two techniques aim to enhance the realism of latent vectors, labeled as *inverted* and *optimized*, respectively, in contrast to the typical random sampling approach, labeled as *random*. The histogram (left) shows the confidence distribution for ‘truck’ class across 100 synthetic images generated from corresponding latent vectors obtained by each method, compared to the confidence distribution of real data. Each prediction is made by a pretrained oracle classifier. The quality differences between methods are clearly observed not only in the histogram but also from ‘truck’ images (right), each of which is taken as the *median* sample in its confidence distribution (i.e., the sample that ranks 50th in ‘truck’ confidence among 100 images). The graph to the right of the images shows the top-3 confidence values for each image across 10 classes. There’s a substantial gap between the ‘truck’ confidence (i.e., the top-1 confidence value of each sample) and the confidence values of other classes in *inverted* and *optimized*, highlighting the effectiveness of our two techniques.

2016) to GANs, which identifies and tries not to change important parameters from the previous version. Lifelong-GAN (Zhai et al., 2019) exploits knowledge distillation (Hinton et al., 2015) when training an image-conditioned and label-conditioned GAN in order for the outputs of the previous GAN to remain the same in the current GAN being trained. Both approaches are widely used in non-generative CL, as they are effective at preserving discriminative knowledge. However, they turn out to be not the best fit for GCL over GANs where we have to preserve and keep learning generative knowledge.

Expansion-Based. Methods based on network expansion (Cong et al., 2020; Varshney et al., 2021; Verma et al., 2021) continuously increase the GAN’s capacity by incorporating additional parameters for each new task. These methods rarely update the existing parameters that have been trained for previous tasks. Consequently, they offer the benefit of reduced forgetting of earlier knowledge, but demand a continual growth in memory during training and they should retain these additional parameters during inference, potentially leading to an increase in inference time.

Replay-Based. There are two major branches within replay-based methods, namely generative replay and memory replay. In order to preserve previous knowledge, generative replay exploits synthetic data generated by the GAN itself, which has been trained up to that point. DGR (Shin et al., 2017) and MeRGAN (Wu et al., 2018) are such examples of adopting generative replay to provide synthetic samples that resemble the previously learned data, with a non-conditional GAN and a conditional GAN (Odena et al., 2017), respectively. These generative replay methods are scalable, as the GAN being trained is the sole component requiring memory storage regardless of the length of the tasks. However, their performance is not satisfactory to the point that they are only effective on simple tasks such as MNIST, where the generated images are not considerably different from real images (Lesort et al., 2019a). On the other hand, memory replay utilizes memory for storing real data samples, and thereby achieving relatively better performance than generative replay in non-generative CL. In generative CL, CloGAN (Rios & Itti, 2019) applies memory replay to replay stored real images together with CFM (Class-Conditioned Filtering) that filters synthetic images generated from the current generator being trained. To enrich the diversity of samples in memory, it employs a clustered distribution of real data to select more representative images. However, a few representative images cannot effectively capture the whole distribution of the previous data, leading to limited performance in our experimental results. Our approach is designed to overcome the limitations inherent to both generative and memory replay strategies through two key aspects: how to use the limited memory buffer more efficiently for knowledge preservation than the current memory replay scheme does, and how to boost the quality of samples that are generated by the GAN, compared to the vanilla generative replay method.

Generative CL using Latent Vectors of VAEs. Similar to GANs, VAEs have also served as a provider of synthetic samples for previous tasks in CL. Some of the VAE-based CL methods naturally utilize latent vectors, as they are intermediate representations in the middle of the encoder and decoder of a VAE network. One example is MIR (Aljundi et al., 2019), which optimizes latent vectors to generate most interfered images from the perspective of a new task. These generated images are then used to replay previous knowledge in a classification task. Similarly, Riemer et al. (2019) explore the usage of a discrete VAE, wherein discrete latent codes corresponding to real images are stored. In contrast to these methods using latent vectors, this work focuses on the fact that the GAN’s input vectors have been used in a too straightforward way in the context of GCL, mostly being chosen at random. Drawing inspiration from the above VAE-based methods, this paper suggests taking the best use of input vectors of the GAN, which would need quite novel techniques due to the differences between GANs and VAEs. Notably, input vectors of GANs are neither trivially obtainable nor properly adjustable, unlike VAEs relying on trainable (i.e., adaptable) encoders that directly produces latent vectors. To the best of our knowledge, this is the first study that deals with GCL using GANs by optimizing or storing latent input vectors to enhance the sample quality.

Feature Replay in Non-Generative CL. In non-generative CL for classification models, there is an approach of employing feature vectors, called *feature replay*, which replays features instead of images to preserve the past knowledge residing in these features as well. By saving (Pellegrini et al., 2020; Hayes et al., 2020) or generating (Liu et al., 2020; Shen et al., 2021; Van de Ven et al., 2020) intermediate features of a classification model, they have shown the possibility of efficiently using memory or effectively representing the complex distribution of challenging datasets. However, it is noteworthy that these intermediate features are different from our latent vectors in that they are intermediate output representations resulting from an input image while our latent vectors are the inputs themselves to the generator, from which we create the corresponding synthetic images.

3 PRELIMINARY

In this section, we present preliminary definitions of GANs along with conditional GANs, and then formulate the problem of generative continual learning over GANs with a focus on replay-based strategies.

3.1 GANS AND CONDITIONAL GANS

A GAN (Goodfellow et al., 2014) is a model that takes a low-dimensional input vector z randomly sampled from a particular distribution $p_z(z)$ (e.g., Gaussian) and produces a high-dimensional image corresponding to z . Training a GAN involves alternately updating its generator G and discriminator D , aiming to approximately capture the hidden distribution $p_{data}(x)$ of real data. The discriminator D is trained to accurately distinguish fake generated images $G(z)$ from the real ones x , and thus is a binary classifier on two labels, ‘real’ and ‘fake’. On the other hand, the goal of the generator G is to create synthetic images realistic enough to be classified as ‘real’ by D . For training, the standard GAN optimizes:

$$\min_G \max_D V_{GAN}(G, D) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]. \quad (1)$$

Similarly, a conditional GAN (Mirza & Osindero, 2014) generates images at given conditional label y , and is trained by:

$$\min_G \max_D V_{CGAN}(G, D) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x|y)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z|y)))]. \quad (2)$$

For brevity, we describe our problem and methodology with respect to non-conditional GANs using Eq (1), but the same procedure can trivially be applied to conditional GANs.

3.2 GAN-BASED GENERATIVE CONTINUAL LEARNING

In the problem of GAN-based GCL, we are given a task sequence as $T_1, T_2, \dots, T_n$, where T_t is associated with its real dataset $\mathcal{X}_t$ at each time step t . In replay-based methods, memory replay maintains the memory buffer $\mathcal{M}_{t-1}$ of a fixed size, storing a subset of previously observed samples at task T_t , and replays them along with $\mathcal{X}_t$. On the other hand, generative replay retrains the synthesized data reconstructed with the previous generator G_{t-1} from randomly sampled latent vectors. Given a pretrained GAN model $\Theta_{t-1} = (G_{t-1}, D_{t-1})$ and a new task T_t , the objective of GCL for the GAN is to train $\Theta_t = (G_t, D_t)$, which is initialized as Θ_{t-1} , such that G_t can generate realistic yet diverse images resembling all of the previous and current images in $\{\mathcal{X}_1, \mathcal{X}_2, \dots, \mathcal{X}_{t-1}, \mathcal{X}_t\}$. In memory replay, we train Θ_t with real images in $\mathcal{M}_{t-1} \cup \mathcal{X}_t$, while generative replay regards the set $\mathcal{G}_{t-1}$ of synthetic images, i.e., $\mathcal{G}_{t-1} = \{\hat{x} \mid \hat{x} = G_{t-1}(z) \text{ s.t. } z \sim p_z(z)\}$ as real images along with the ones in $\mathcal{X}_t$.

4 METHODS

In this section, we propose our *RactoFit* method for GCL for GANs, which can effectively retain realistic knowledge throughout the continual training process by storing inverted latent vectors in the memory buffer instead of raw images and dynamically optimizing random vectors to boost the level of realism in the generative knowledge to be transferred to the subsequent tasks.

Overall Process. The key idea of our solution is twofold. First, to utilize the memory space in a more efficient way in memory replay, we store the latent vectors corresponding to real images of previous tasks in the buffer, instead of maintaining raw images themselves. Second, we devise a novel optimization technique for latent vectors of a trained GAN, aiming to generate more realistic images than those generated from random vectors. These synthetic images, whose realism are enhanced by either of our techniques, are then replayed with the images of the current task. The whole process of *RactoFit* consists of the following three main steps: (i) constructing the memory buffer for the training of the subsequent task, (ii) optimizing latent vectors to be realistic which are randomly initialized, and (iii) training a given new task while replaying a union of the stored vectors and dynamically optimized vectors.

Algorithm 1 and Figure 2 present the detailed training procedure of *RactoFit*. For the first task T_1 (Lines 8-9), we conduct the typical GAN training by minimizing the loss function Eq. (1) with solely $\mathcal{X}_1$, while training of each subsequent task T_t for $t > 1$ (Lines 10-17) takes a joint set of (i) real images in $\mathcal{X}_t$, (ii) reconstructed images of latent vectors in the buffer $\mathcal{Z}_{t-1}$, and (iii) generated images from realistically optimized latent vectors in $\mathcal{Z}_{opt}$, to fully capture the entire data distribution of all the previous tasks as well as the current task.

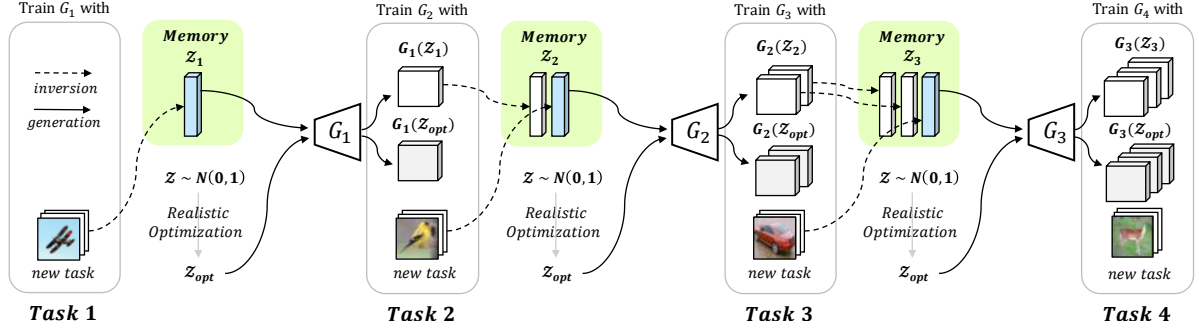


Figure 2: Overall training process of the proposed RactoFit method. At task T_1 , we train G_1 with the first task (airplane) and then save the inverted latent vectors of a subset of these images in memory. At task T_2 , we train G_2 with the images from the box of T_2 , which is a combination of (i) reconstructed images of latent vectors in memory $G_1(Z_1)$, (ii) generated images from realistically optimized latent vectors $G_1(Z_{opt})$, and (iii) new task images (bird), presented in order from the top. After training, we update the latent vectors from the previous memory and newly invert the new images. This process is continuously repeated.

Algorithm 1 RactoFit ($\Theta_0 = (G_0, D_0)$, $\langle T_1, T_2, \dots, T_n \rangle$)

```

1:  $\Theta_0 \leftarrow$  a GAN randomly initialized
2:  $Z_0 \leftarrow \emptyset$ 
3: for each  $T_t$ , where  $t \in [1, n]$  do
4:   Given:  $\Theta_{t-1} = (G_{t-1}, D_{t-1})$ ,  $Z_{t-1}$ ,  $\mathcal{X}_t$ 
5:   Output:  $\Theta_t = (G_t, D_t)$ 
6:    $\Theta_t \leftarrow \Theta_{t-1}$ 
7:   if  $t = 1$  then
8:     Train  $\Theta_1$  with  $\mathcal{X}_1$  by Eq. (1)
9:      $Z_1 \leftarrow$  invert subset of  $\mathcal{X}_1$  w.r.t.  $G_1$  by Eq. (4)
10:  else
11:     $Z_{opt} \leftarrow \{z \mid z \sim \mathcal{N}(0, 1)\}$ 
12:     $Z_{opt} \leftarrow$  Optimizing  $Z_{opt}$  by Eq. (6)
13:    Train  $\Theta_t$  with  $G_{t-1}(Z_{t-1} \cup Z_{opt}) \cup \mathcal{X}_t$  by Eq. (1)
14:    Update  $Z_{t-1}$  to  $Z_t$  with Eq. (3)
15:     $Z_{new} \leftarrow$  invert subset of  $\mathcal{X}_t$  w.r.t.  $G_{t-1}$  by Eq. (4)
16:     $Z_t \leftarrow Z_{t-1} \cup Z_{new}$ 
17:  end if
18: end for

```

4.1 BUFFER CONSTRUCTION

Once the training process of Θ_t is done, we construct the memory buffer Z_t to be given for training Θ_{t+1} . The buffer construction process consists of the following two steps. First, we need to replace the latent vectors from the current buffer Z_{t-1} with the updated ones according to the newly trained G_t , thereby obtaining the initial version of Z_t . Then, for some real images chosen from $\mathcal{X}_t$, we convert them into their corresponding latent vectors with respect to G_t , and incorporate these new vectors into Z_t as well.

Updating Previous Latent Vectors. Since the latent vectors in Z_{t-1} would have been placed in the embedding space of the previous generator G_{t-1} , it is mandatory to update these vectors to be mapped in the transformed space of the newly trained generator G_t . Otherwise, the latent vector of an image at some point would no longer yield the same image after training a few subsequent tasks. Figure 3 illustrates such an example, where we train a sequence of MNIST tasks, one for each digit. At the end of training the first task on ‘0’ images, we invert some of these training images into the corresponding latent vectors using G_1 , but these vectors are mapped to totally different images with generators newly trained for a sequence of digits ‘1’, ‘2’, ‘3’, etc. To avoid this situation, we update each latent vector z_{t-1} from Z_{t-1} inverted using G_{t-1} to the vector z_t inverted with respect to the current generator G_t by:

$$z_t = \underset{z}{\operatorname{argmin}} \mathcal{L}_{inv}(G_{t-1}(z_{t-1}), G_t(z)), \quad (3)$$

where $\mathcal{L}_{inv}(\cdot, \cdot)$ is a GAN inversion loss that quantifies difference between the target input image and the reconstructed image from the latent vector being trained. Thus, in Eq. (3), the target image is the synthetic image generated from

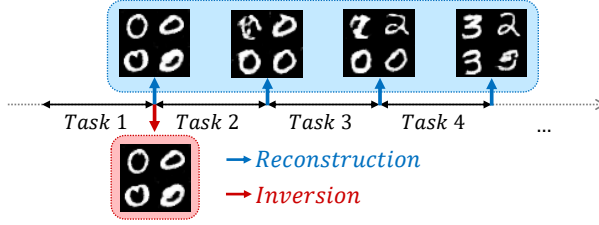


Figure 3: An illustrative example of the continual deterioration of quality of images from the latent vectors that are inverted at the first task on MNIST.

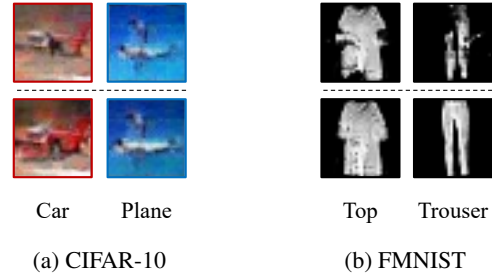


Figure 4: Effectiveness of the proposed realistic optimization technique, where images generated from random vectors (*top*) or their corresponding optimized vectors (*bottom*) in CIFAR-10 and FMNIST.

G_{t-1} , and our objective is to minimize the difference of the newly generated image $G_t(\mathbf{z})$ from its target image. All these updated vectors initially constitute $\mathcal{Z}_t$.

Inverting the Current Data. Our next step is to convert a particular number of real images chosen from $\mathcal{X}_t$ into their corresponding latent vectors with respect to G_t , which are also included in $\mathcal{Z}_t$. For each real image $\mathbf{x} \in \mathcal{X}_t$, we can optimize the following function to obtain its inverted latent vector, similar to Eq. (3):

$$\mathbf{z}_t = \underset{\mathbf{z}}{\operatorname{argmin}} \mathcal{L}_{inv}(\mathbf{x}, G_t(\mathbf{z})). \quad (4)$$

By incorporating all these vectors for the current task into $\mathcal{Z}_t$, we complete the buffer construction for T_t .

It is worthwhile to note that as long as we use the same amount of memory space as an image buffer $\mathcal{M}_t$ adopted in typical memory replay, it holds that $|\mathcal{Z}_t| \gg |\mathcal{M}_t|$ at any time step t due to the size difference between vectors and images. For example, either our buffer with $|\mathcal{Z}_t| = 2400$ or the image buffer with $|\mathcal{M}_t| = 100$ use the same buffer space in our experiments using CIFAR-10. This enables our buffer strategy to capture a wide range of knowledge about the underlying data distribution.

Inversion Loss Function.

As a GAN inversion loss function, we adopt L1 or L2 Norm based per-pixel distance combined with LPIPS metric (Zhang et al., 2018; Huh et al., 2020) as follows:

$$\mathcal{L}_{inv}(\mathbf{x}, \hat{\mathbf{x}}) = \|\mathbf{x}, \hat{\mathbf{x}}\|_d + \beta \mathcal{L}_{LPIPS}(\mathbf{x}, \hat{\mathbf{x}}), \quad (5)$$

where $\mathbf{x}$ is the target image and $\hat{\mathbf{x}}$ is the generated image.

4.2 REALISTIC OPTIMIZATION

Despite the fact that we can store plenty of vectors that correspond to real images as $|\mathcal{Z}_t| \gg |\mathcal{M}_t|$, training a long sequence of tasks still requires *pure* synthetic images, which are randomly generated, to fully capture the complex data distribution as in generative replay. Based on this motivation, we propose a realistic optimization technique that can boost the level of realism of such synthetic images. To this end, we focus on the fact that the discriminator, which has been trained and used for its corresponding GAN for a specific task, is indeed a learned prior of the real data distribution for the task (Yi et al., 2018). Therefore, in the optimization process, we use the discriminator to quantify to what extent each synthetic image is realistic with respect to the tasks learned so far.

More specifically, for each randomly initialized vector $\mathbf{z}$, we optimize $\mathbf{z}$ itself for its corresponding image to get a higher score from the previous discriminator D_{t-1} by:

$$\mathbf{z}^* = \underset{\mathbf{z}}{\operatorname{argmax}} D_{t-1}(G_{t-1}(\mathbf{z})). \quad (6)$$

As visually observed in Figure 4, this realistic optimization is quite effective to make proper adjustment for each random vector so that the resulting generated image becomes more realistic and clearer in FMNIST and CIFAR-10.

It is important to note that our realistic optimization does not hurt the diversity of generated images, and it rather enhances the diversity according to our empirical study. To investigate, we measure the *precision* and *recall* (Kynkäänniemi et al., 2019) values, obtained from a total of 10k images on CIFAR-10, with 1k images per class. Given that recall is a measure of diversity, we can clearly observe that the distribution of generated images becomes more diverse through our realistic optimization process.

Table 1: Diversity analysis of Realistic Optimization on CIFAR-10

Methods	Precision	Recall
Random	0.790	0.182
w/ Realistic Optimization	0.790	0.193

5 EXPERIMENTS

5.1 EXPERIMENTAL SETTINGS

Datasets and Models. In our experiments, we use four benchmark datasets commonly used for evaluating GCL methods, namely MNIST (LeCun et al., 1998), FMNIST (Xiao et al., 2017), SVHN (Netzer et al., 2011), and CIFAR-10 (Krizhevsky et al., 2009). MNIST and FMNIST each have 60k training images and 10k test images, whereas CIFAR-10 includes 50k training images and 10k test images, all distributed across 10 classes. The images in MNIST and FMNIST are sized at 28×28 , while those in CIFAR-10 are 32×32 . In the context of GCL, CIFAR-10 is considered relatively complex, compared to the other datasets. For all benchmark datasets, we construct a sequence of 10 tasks, each corresponding to a single class. The GAN architecture employed for both MNIST and FMNIST is based on DCGAN (Radford et al., 2016), featuring two fully connected layers and two convolutional layers in both the generator and discriminator. In CGANs conditional on each class, input images as well as latent vectors are multiplied by their corresponding embedding vectors in MNIST, while one-hot encoded vectors are concatenated with them and fed into CGANs for FMNIST. In the case of CIFAR-10, the model consists of one fully connected layer and three convolutional layers in both its generator and discriminator. The condition label is embedded and multiplied as in MNIST. Also, we use the Wasserstein loss (Arjovsky et al., 2017) with gradient penalty (Gulrajani et al., 2017) for training CIFAR-10. Further details and results for SVHN are available in the Appendix.

Implementation and Training Details. We use PyTorch for implementing our methods and train the models using NVIDIA Quadro RTX 6000. Full details are covered in the Appendix.

Evaluation Metrics. To evaluate the performance of all methods, we employ standard quantitative metrics for GANs, namely Fréchet Inception Distance (FID) (Heusel et al., 2017) and fitting capacity (Lesort et al., 2019b). FID aims to assess the dissimilarity between the distribution of generated samples and the real data distribution. To this end, we construct two Gaussian models on features, one for generated images and the other for real images, assuming both distributions follow multivariate Gaussian patterns. The features required for these models can be obtained through a pretrained model, for which the Inception-v3 network (Szegedy et al., 2016) trained on ImageNet (Deng et al., 2009) has been widely used. Instead of Inception-v3, we employ *expert* networks with two convolutional layers, which are pretrained on each specific dataset, aligning with the approach of previous works (Li et al., 2017; Lesort et al., 2019b). This choice is motivated by the fact that each benchmark dataset has its own distinct distribution that is also significantly different from that of ImageNet. Utilizing a model pretrained on ImageNet to compute FID for those datasets may lead to inaccurate results misaligned with human judgement as extensively reported in the literature (Kynkäänniemi et al., 2022; Morozov et al., 2021). Fitting capacity, on the other hand, assesses the generative model’s ability to accurately replicate the distribution of the original data by measuring the accuracy on real data using a classifier trained with generated samples. During the classifier training process, labeled images are obtained directly from CGANs while labels are derived from the corresponding expert model for non-conditional GANs. We use the same architecture for both the expert model and the classifier, which is based on a model with two convolutional layers for MNIST and FMNIST, and ResNet18 (He et al., 2016) for CIFAR-10.

Compared Methods. We compare our RactoFit method with the following baseline approaches:

- **Upperbound:** This method represents the upper bound, where the model is sequentially yet jointly trained using the complete set of either previous or new images.
- **EWC (Kirkpatrick et al., 2016):** This is a well-known parameter regularization method widely applied in various CL problems.
- **MeRGAN (Wu et al., 2018)** (for CGANs) or **DGR (Shin et al., 2017)** (for GANs): This baseline involves a typical generative replay method without using any buffered images.
- **Rehearsal:** This approach is memory replay that trains the model with a subset of previous images stored in the buffer.
- **CloGAN (Rios & Itti, 2019):** This can be seen as a type of mixed replay where memory replay is combined with generative replay, incorporating additional filtering techniques.
- **MeRGAN or DGR + Rehearsal:** This intends to test a straightforward approach of mixing memory replay with generative replay without any extra techniques.

Memory Buffer. Except for EWC and MeRGAN (or DGR), all the compared methods, including RactoFit, make use of a memory buffer. For a fair comparison, we allocate an equivalent memory space to these methods as well as RactoFit. For all datasets, we set the buffer size proportionally to 0.8% (e.g., 500 images, 50 images per task) and

Table 2: GCL performance comparison upon the completion of all continual tasks on MNIST and FMNIST.

Methods/ Dataset	Buffer (%) ($ \mathcal{M} : \mathcal{Z} $)	MNIST				FMNIST			
		GAN		CGAN		GAN		CGAN	
		FID (expert)	fitting capacity	FID (expert)	fitting capacity	FID (expert)	fitting capacity	FID (expert)	fitting capacity
Upperbound	100%	0.70 $\pm$ 0.44	98.06 $\pm$ 0.15	0.80 $\pm$ 0.31	97.97 $\pm$ 0.12	0.41 $\pm$ 0.03	85.35 $\pm$ 0.39	0.93 $\pm$ 0.03	80.90 $\pm$ 0.33
EWC	0%	26.35 $\pm$ 6.05	76.32 $\pm$ 12.71	34.19 $\pm$ 3.50	17.24 $\pm$ 3.90	18.72 $\pm$ 3.47	35.32 $\pm$ 5.61	23.54 $\pm$ 0.31	10.86 $\pm$ 1.26
DGR/MeRGAN		2.61 $\pm$ 0.39	97.57 $\pm$ 0.18	2.34 $\pm$ 0.22	96.42 $\pm$ 0.49	1.22 $\pm$ 0.07	84.15 $\pm$ 0.25	1.71 $\pm$ 0.05	74.95 $\pm$ 0.71
RactoFit (ours)		2.85 $\pm$ 0.40	97.05 $\pm$ 0.23	2.55 $\pm$ 0.30	96.17 $\pm$ 0.63	1.20 $\pm$ 0.12	83.01 $\pm$ 0.40	2.08 $\pm$ 0.22	76.86 $\pm$ 1.35
Rehearsal	0.8% (0.5K : 7K)	36.30 $\pm$ 1.15	71.05 $\pm$ 1.96	36.26 $\pm$ 1.07	44.17 $\pm$ 3.87	21.08 $\pm$ 0.84	67.25 $\pm$ 3.54	3.11 $\pm$ 0.15	71.84 $\pm$ 1.30
DGR/MeRGAN + Rehearsal		2.67 $\pm$ 0.42	97.70 $\pm$ 0.17	2.16 $\pm$ 0.11	96.52 $\pm$ 0.60	1.17 $\pm$ 0.10	83.86 $\pm$ 0.32	1.97 $\pm$ 0.23	76.07 $\pm$ 2.06
CloGAN		-	-	13.79 $\pm$ 1.50	91.67 $\pm$ 1.65	-	-	6.66 $\pm$ 1.38	64.48 $\pm$ 4.82
RactoFit (ours)		1.92 $\pm$ 0.25	97.45 $\pm$ 0.24	1.82 $\pm$ 0.16	97.06 $\pm$ 0.18	0.96 $\pm$ 0.10	84.01 $\pm$ 0.51	1.85 $\pm$ 0.18	76.78 $\pm$ 1.08
Rehearsal	1.6% (1K : 14K)	30.11 $\pm$ 1.02	88.91 $\pm$ 2.38	37.07 $\pm$ 3.28	39.27 $\pm$ 4.50	17.68 $\pm$ 0.57	70.81 $\pm$ 3.55	2.55 $\pm$ 0.13	72.96 $\pm$ 0.90
DGR/MeRGAN + Rehearsal		2.01 $\pm$ 0.37	97.92 $\pm$ 0.11	2.35 $\pm$ 0.23	96.46 $\pm$ 0.59	1.06 $\pm$ 0.06	83.93 $\pm$ 0.49	1.81 $\pm$ 0.08	77.09 $\pm$ 0.90
CloGAN		-	-	10.65 $\pm$ 0.79	94.22 $\pm$ 0.41	-	-	5.68 $\pm$ 1.20	69.39 $\pm$ 5.88
RactoFit (ours)		1.96 $\pm$ 0.28	97.38 $\pm$ 0.28	1.43 $\pm$ 0.11	97.46 $\pm$ 0.20	0.88 $\pm$ 0.06	84.35 $\pm$ 0.61	1.52 $\pm$ 0.11	77.66 $\pm$ 0.75

Table 3: GCL performance comparison upon the completion of all continual tasks on CIFAR-10.

CIFAR-10			
Methods	Buffer (%) ($ \mathcal{M} : \mathcal{Z} $)	FID (expert)	fitting capacity
Upperbound	100%	0.85 $\pm$ 0.07	57.43 $\pm$ 1.89
EWC	0%	2.72 $\pm$ 0.27	15.31 $\pm$ 0.85
MeRGAN		4.40 $\pm$ 0.22	27.90 $\pm$ 2.69
RactoFit (ours)		1.57 $\pm$ 0.12	36.69 $\pm$ 1.76
Rehearsal	0.8% (0.5K : 12K)	3.06 $\pm$ 0.12	33.80 $\pm$ 0.95
MeRGAN + Rehearsal		3.84 $\pm$ 0.24	28.82 $\pm$ 1.73
CloGAN		2.40 $\pm$ 0.43	21.14 $\pm$ 2.78
RactoFit (ours)		0.87 $\pm$ 0.08	38.47 $\pm$ 1.54
Rehearsal	1.6% (1K : 24K)	2.83 $\pm$ 0.16	38.27 $\pm$ 1.69
MeRGAN + Rehearsal		4.06 $\pm$ 0.55	29.44 $\pm$ 2.51
CloGAN		2.27 $\pm$ 0.27	22.13 $\pm$ 4.17
RactoFit (ours)		0.68 $\pm$ 0.03	38.33 $\pm$ 2.25

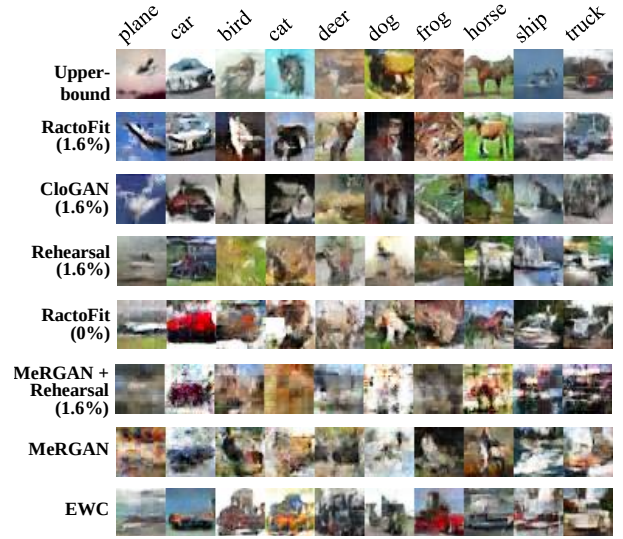


Figure 5: Generated samples from each method after continual learning of all tasks on CIFAR-10.

(e.g., 1,000 images, 100 images per task) of the total number of images (e.g., 60,000) in the dataset. Given the difference in dimensions between each image and its latent vector, we adjust the number of images or vectors to be stored in the buffer according to the ratio of the image size to the vector size. In the case of MNIST and FMNIST, each image is in 784 dimensions but its latent vector is only 56 dimensions, allowing us to store 14 times more vectors than images within the same memory space. For CIFAR-10, this ratio is 1/24 as images and vectors are in 3072 dimensions and 128 dimensions, respectively. While it is possible to compress the original image without significant loss of quality, we have not considered any compression methods in this paper.

5.2 EXPERIMENTAL RESULTS

5.2.1 OVERALL PERFORMANCE

Tables 2 and 3 present a quantitative performance comparison upon the completion of training for all continual tasks, where we report the average values along with their standard deviation over five runs with different seeds. In both tables, methods are categorized based on their buffer sizes of 0%, 0.8%, and 1.6%, respectively.

Performance in MNIST and FMNIST. In Table 2 showing the results using MNIST and FMNIST, RactoFit is clearly the best method in terms of both quality metrics. RactoFit also shows efficiency in memory utilization, consistently outperforming Rehearsal (i.e., memory replay), even with a smaller memory space. For example, the quality of RactoFit (0.8%) surpasses that of Rehearsal (1.6%) across all cases. This confirms the limitation of memory replay in

capturing sufficient knowledge about previous tasks without storing a substantial number of real images. Combining memory replay with generative replay (DGR or MeRGAN) improves performance, still not as satisfactory as our RactoFit method. Both EWC and CloGAN exhibit low performance, possibly because they are not precisely designed for our GCL problem setting. In particular, CloGAN focuses on improving the classification performance of AC-GAN (Odena et al., 2017), explaining its lower FID performance compared to fitting capacity. DGR or MeRGAN demonstrate reasonable performance, comparable to CloGAN and occasionally better than RactoFit without using any buffer space. This reveals the efficacy of generative replay in capturing simple data distributions, such as MNIST, using abundant synthetic images, even though there is still room for improvement as RactoFit surpasses generative replay when using only a small buffer space.

Performance in CIFAR-10. Table 3 shows the GCL performance using CIFAR-10. As mentioned earlier, CIFAR-10 is usually considered a challenging dataset within the GCL context (Lesort et al., 2019a), and hence MeRGAN as well as MeRGAN with Rehearsal faces difficulties in maintaining high performance. This outcome contrasts with the GCL results for MNIST and FMNIST in Table 2, where MeRGAN performs nearly as well as the upper bound. This discrepancy suggests that generative replay encounters challenges due to the gap between true real images and synthetic images labeled as real. On the other hand, RactoFit continues to be successful in preserving realistic generative knowledge across tasks, thereby always achieving the best FID and fitting capacity values. Even without any memory space, RactoFit (0%) manages to perform better than Rehearsal (0.8%).

Qualitative Result. Figure 5 presents the generated images from each method after the completion of training all 10 tasks of CIFAR-10. In each method, the columns represent the 10 classes in CIFAR-10. EWC mainly generates images of the final class ‘truck’, indicating a tendency to forget previous knowledge. MeRGAN and MeRGAN + Rehearsal (1.6%) produce rather noisy images, which means they fail to transfer realistic knowledge across tasks. CloGAN (1.6%) generates relatively clear images, but those from later tasks like ‘horse’, ‘ship’ and ‘truck’ are difficult to recognize. In contrast, our RactoFit (1.6%) method generates more realistic images throughout the 10 tasks by preserving the quality of previous tasks.

Ablation Study. To analyze the effectiveness of each component in RactoFit, namely Buffered Vectors (BV) and Realistic Optimization (RO), we conduct an ablation study by removing one or both components and examining the resulting performance. As outlined in Table 4, both BV and RO are essential to achieve the best performance of RactoFit, as the removal of both leads to about an increase of 3.71 in FID and a reduction of 11.09 in fitting capacity. Somewhat interestingly, it turns out that RO alone makes a bit more impact than BV itself, possibly due to the relatively small 0.8% buffer space. In conclusion, however, the synergy effect of using both RO and BV is clearly observed, as their individual utilization ends up with much lower performance.

Table 4: Ablation study to analyze the impact of each component of RactoFit, namely BV (Buffered Vectors) and RO (Realistic Optimization), with 0.8% memory buffer on CIFAR-10.

Methods	FID (expert)	fitting capacity
RactoFit	0.84	37.80
RactoFit w/o BV	1.72	35.14
RactoFit w/o RO	2.98	27.01
RactoFit w/o BV and RO	4.55	26.71

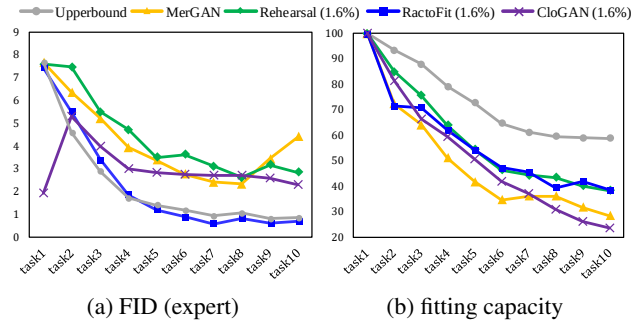


Figure 6: GCL performance after the training process for each continual task on CIFAR-10.

5.2.2 TASK INCREMENTAL PERFORMANCE

Figure 6 illustrates how performance changes during each incremental learning phase for a given task on CIFAR-10. After training with task-specific data at the time step t , we report FID scores and fitting capacity values obtained for all the tasks $T_1, T_2, \dots, T_t$. RactoFit consistently maintains high performance during the incremental process of GCL, whereas MerGAN’s FID scores sharply spike at task 8 in Figure 6a, indicating a degradation of realism for generated images that accumulates to a point where the GAN struggles to learn from such unrealistic synthesized images labeled as real. In Figure 6b, we observe that RactoFit achieves the best fitting capacity scores in latter tasks (task 9, 10) highlighting its efficacy in mitigating forgetting by effectively extracting and transferring more realistic knowledge across incremental steps.

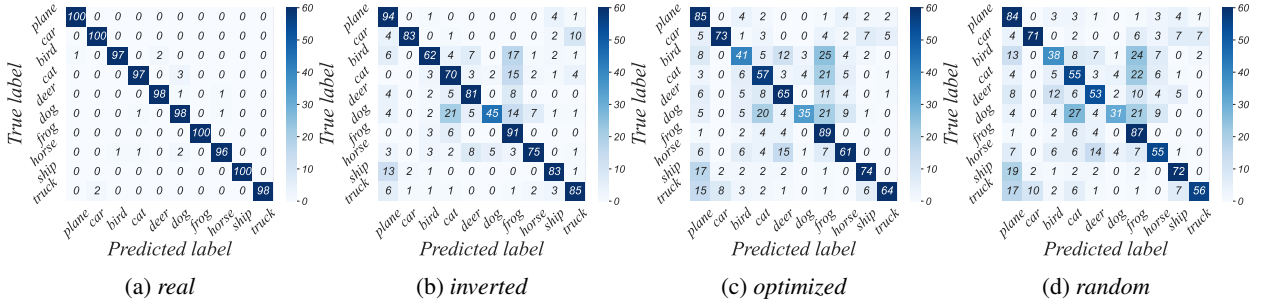


Figure 7: Confusion matrices on predictions of ViT with real or synthetic images generated by each method in CIFAR-10.

5.2.3 EFFECTIVENESS OF REALISTIC KNOWLEDGE EXTRACTION

We also examine the effectiveness of our *inverted* (by Eq. (4)) or *optimized* (by Eq. (6)) latent vectors in extracting realistic knowledge from a trained GAN. For this purpose, we employ the upper bound in Table 3 as a well-trained GAN and an oracle ViT classifier (Dosovitskiy et al., 2021) with 98.35% accuracy on CIFAR-10. We then compare the classification results of the oracle model on images generated from our proposed techniques with those of *real* data and synthetic data generated from *random* vectors. The underlying intuition of this experimental setup is how well a human-level AI model (i.e., oracle) can recognize each synthetic image.

As highlighted in Figure 1, we clearly observe that images synthesized using inverted vectors are recognized by the oracle with the highest confidence, and optimized vectors are also effective in enhancing the confidence of resulting images, compared to random vectors. Figure 7 presents confusion matrices on predicted classes by the oracle using real or synthesized images. In this analysis, our inverted or optimized vectors effectively reduce misclassification counts by boosting the realism of generated images. Based on these results, we can conclude that the latent vectors obtained by our proposed techniques can carry more realistic knowledge from a trained GAN, which is a crucial factor contributing to the superior GCL performance of RactoFit.

6 CONCLUSION

In replay-based GCL for GANs, we proposed a mixed replay method, *RactoFit*, which enjoys the benefits of both generative and memory replay by making the best use of the input latent vectors of GANs. Our first suggestion is to store inverted latent vectors of real images, thereby improving the memory utilization. We then devised a realistic optimization technique for random latent vectors. Through both strategies, RactoFit is capable of extracting and transferring highly realistic knowledge across continual steps, outperforming the baseline methods in GCL benchmark tests.

ACKNOWLEDGMENTS

This work was supported in part by Institute of Information & communications Technology Planning & Evaluation (IITP) grants funded by the Korea government(MSIT) (No.2022-0-00448, Deep Total Recall: Continual Learning for Human-Like Recall of Artificial Neural Networks, No.RS-2022-00155915, Artificial Intelligence Convergence Innovation Human Resources Development (Inha University)), and in part by the National Research Foundation of Korea (NRF) grants funded by the Korea government (MSIT) (No.2022R1A4A3029480).

REFERENCES

- Rahaf Aljundi, Lucas Caccia, Eugene Belilovsky, Massimo Caccia, Min Lin, Laurent Charlin, and Tinne Tuytelaars. Online continual learning with maximally interfered retrieval. *CoRR*, abs/1908.04742, 2019. URL <http://arxiv.org/abs/1908.04742>.
- Martín Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein GAN. *CoRR*, abs/1701.07875, 2017. URL <http://arxiv.org/abs/1701.07875>.

- Andrew Brock, Jeff Donahue, and Karen Simonyan. Large scale GAN training for high fidelity natural image synthesis. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019. URL <https://openreview.net/forum?id=B1xsqj09Fm>.
- Yulai Cong, Miaoyun Zhao, Jianqiao Li, Sijia Wang, and Lawrence Carin. GAN memory with no forgetting. In Hugo Larochelle, Marc’Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (eds.), *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/bf201d5407a6509fa536afc4b380577e-Abstract.html>.
- Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR 2009), 20-25 June 2009, Miami, Florida, USA*, pp. 248–255. IEEE Computer Society, 2009. doi: 10.1109/CVPR.2009.5206848. URL <https://doi.org/10.1109/CVPR.2009.5206848>.
- Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. URL <https://openreview.net/forum?id=YicbFdNTTy>.
- Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron C. Courville, and Yoshua Bengio. Generative adversarial networks. *CoRR*, abs/1406.2661, 2014. URL <http://arxiv.org/abs/1406.2661>.
- Ishaan Gulrajani, Faruk Ahmed, Martín Arjovsky, Vincent Dumoulin, and Aaron C. Courville. Improved training of wasserstein gans. In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (eds.), *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pp. 5767–5777, 2017. URL <https://proceedings.neurips.cc/paper/2017/hash/892c3b1c6dccc52936e27cbd0ff683d6-Abstract.html>.
- Tyler L. Hayes, Kushal Kafle, Robik Shrestha, Manoj Acharya, and Christopher Kanan. REMIND your neural network to prevent catastrophic forgetting. In Andrea Vedaldi, Horst Bischof, Thomas Brox, and Jan-Michael Frahm (eds.), *Computer Vision - ECCV 2020 - 16th European Conference, Glasgow, UK, August 23-28, 2020, Proceedings, Part VIII*, volume 12353 of *Lecture Notes in Computer Science*, pp. 466–483. Springer, 2020. doi: 10.1007/978-3-030-58598-3_28. URL https://doi.org/10.1007/978-3-030-58598-3_28.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*, pp. 770–778. IEEE Computer Society, 2016. doi: 10.1109/CVPR.2016.90. URL <https://doi.org/10.1109/CVPR.2016.90>.
- Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (eds.), *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pp. 6626–6637, 2017. URL <https://proceedings.neurips.cc/paper/2017/hash/8a1d694707eb0fefe65871369074926d-Abstract.html>.
- Geoffrey E. Hinton, Oriol Vinyals, and Jeffrey Dean. Distilling the knowledge in a neural network. *CoRR*, abs/1503.02531, 2015. URL <http://arxiv.org/abs/1503.02531>.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In Hugo Larochelle, Marc’Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (eds.), *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/4c5bcfec8584af0d967f1ab10179ca4b-Abstract.html>.
- Minyoung Huh, Richard Zhang, Jun-Yan Zhu, Sylvain Paris, and Aaron Hertzmann. Transforming and projecting images into class-conditional generative networks. In Andrea Vedaldi, Horst Bischof, Thomas Brox, and Jan-Michael Frahm (eds.), *Computer Vision - ECCV 2020 - 16th European Conference, Glasgow, UK, August 23-28,*

- 2020, *Proceedings, Part II*, volume 12347 of *Lecture Notes in Computer Science*, pp. 17–34. Springer, 2020. doi: 10.1007/978-3-030-58536-5_2. URL https://doi.org/10.1007/978-3-030-58536-5_2.
- Minguk Kang, Jun-Yan Zhu, Richard Zhang, Jaesik Park, Eli Shechtman, Sylvain Paris, and Taesung Park. Scaling up gans for text-to-image synthesis. *CoRR*, abs/2303.05511, 2023. doi: 10.48550/arXiv.2303.05511. URL <https://doi.org/10.48550/arXiv.2303.05511>.
- Tero Karras, Samuli Laine, and Timo Aila. A style-based generator architecture for generative adversarial networks. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2019, Long Beach, CA, USA, June 16-20, 2019*, pp. 4401–4410. Computer Vision Foundation / IEEE, 2019. doi: 10.1109/CVPR.2019.00453. URL http://openaccess.thecvf.com/content_CVPR_2019/html/Karras_A_Style-Based_Generator_Architecture_for_Generative_Adversarial_Networks_CVPR_2019_paper.html.
- Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In Yoshua Bengio and Yann LeCun (eds.), *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Conference Track Proceedings*, 2014. URL <http://arxiv.org/abs/1312.6114>.
- James Kirkpatrick, Razvan Pascanu, Neil C. Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *CoRR*, abs/1612.00796, 2016. URL <http://arxiv.org/abs/1612.00796>.
- Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.
- Tuomas Kynkäänniemi, Tero Karras, Samuli Laine, Jaakko Lehtinen, and Timo Aila. Improved precision and recall metric for assessing generative models. In Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d’Alché-Buc, Emily B. Fox, and Roman Garnett (eds.), *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pp. 3929–3938, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/0234c510bc6d908b28c70ff313743079-Abstract.html>.
- Tuomas Kynkäänniemi, Tero Karras, Miika Aittala, Timo Aila, and Jaakko Lehtinen. The role of imagenet classes in fréchet inception distance. *CoRR*, abs/2203.06026, 2022. doi: 10.48550/arXiv.2203.06026. URL <https://doi.org/10.48550/arXiv.2203.06026>.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proc. IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791. URL <https://doi.org/10.1109/5.726791>.
- Timothée Lesort, Hugo Caselles-Dupré, Michaël Garcia Ortiz, Andrei Stoian, and David Filliat. Generative models from the perspective of continual learning. In *International Joint Conference on Neural Networks, IJCNN 2019 Budapest, Hungary, July 14-19, 2019*, pp. 1–8. IEEE, 2019a. doi: 10.1109/IJCNN.2019.8851986. URL <https://doi.org/10.1109/IJCNN.2019.8851986>.
- Timothée Lesort, Andrei Stoian, Jean-François Goudou, and David Filliat. Training discriminative models to evaluate generative ones. In Igor V. Tetko, Vera Kurková, Pavel Karpov, and Fabian J. Theis (eds.), *Artificial Neural Networks and Machine Learning - ICANN 2019: Image Processing - 28th International Conference on Artificial Neural Networks, Munich, Germany, September 17-19, 2019, Proceedings, Part III*, volume 11729 of *Lecture Notes in Computer Science*, pp. 604–619. Springer, 2019b. doi: 10.1007/978-3-030-30508-6_48. URL https://doi.org/10.1007/978-3-030-30508-6_48.
- Chunyuan Li, Hao Liu, Changyou Chen, Yunchen Pu, Liqun Chen, Ricardo Henao, and Lawrence Carin. ALICE: towards understanding adversarial learning for joint distribution matching. In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (eds.), *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pp. 5495–5503, 2017. URL <https://proceedings.neurips.cc/paper/2017/hash/ade55409d1224074754035a5a937d2e0-Abstract.html>.
- Xialei Liu, Chenshen Wu, Mikel Menta, Luis Herranz, Bogdan Raducanu, Andrew D. Bagdanov, Shangling Jui, and Joost van de Weijer. Generative feature replay for class-incremental learning. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR Workshops 2020, Seattle, WA, USA, June 14-19, 2020*,

- pp. 915–924. Computer Vision Foundation / IEEE, 2020. doi: 10.1109/CVPRW50498.2020.00121. URL https://openaccess.thecvf.com/content_CVPRW_2020/html/w15/Liu_Generative_Feature_Replay_for_Class-Incremental_Learning_CVPRW_2020_paper.html.
- James L McClelland, Bruce L McNaughton, and Randall C O’Reilly. Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory. *Psychological review*, 102(3):419, 1995.
- Michael McCloskey and Neal J Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of learning and motivation*, volume 24, pp. 109–165. Elsevier, 1989.
- Mehdi Mirza and Simon Osindero. Conditional generative adversarial nets. *CoRR*, abs/1411.1784, 2014. URL <http://arxiv.org/abs/1411.1784>.
- Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net, 2018. URL <https://openreview.net/forum?id=BlQRgziT->.
- Stanislav Morozov, Andrey Voynov, and Artem Babenko. On self-supervised image representations for GAN evaluation. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. URL <https://openreview.net/forum?id=NeRdBeTionN>.
- Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Bo Wu, and Andrew Y Ng. Reading digits in natural images with unsupervised feature learning. 2011.
- Augustus Odena, Christopher Olah, and Jonathon Shlens. Conditional image synthesis with auxiliary classifier gans. In Doina Precup and Yee Whye Teh (eds.), *Proceedings of the 34th International Conference on Machine Learning, ICML 2017, Sydney, NSW, Australia, 6-11 August 2017*, volume 70 of *Proceedings of Machine Learning Research*, pp. 2642–2651. PMLR, 2017. URL <http://proceedings.mlr.press/v70/odena17a.html>.
- Lorenzo Pellegrini, Gabriele Graffieti, Vincenzo Lomonaco, and Davide Maltoni. Latent replay for real-time continual learning. In *IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2020, Las Vegas, NV, USA, October 24, 2020 - January 24, 2021*, pp. 10203–10209. IEEE, 2020. doi: 10.1109/IROS45743.2020.9341460. URL <https://doi.org/10.1109/IROS45743.2020.9341460>.
- Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. In Yoshua Bengio and Yann LeCun (eds.), *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016. URL <http://arxiv.org/abs/1511.06434>.
- Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. Hierarchical text-conditional image generation with CLIP latents. *CoRR*, abs/2204.06125, 2022. doi: 10.48550/arXiv.2204.06125. URL <https://doi.org/10.48550/arXiv.2204.06125>.
- Matthew Riemer, Tim Klinger, Djallel Bouneffouf, and Michele Franceschini. Scalable recollections for continual lifelong learning. In *The Thirty-Third AAAI Conference on Artificial Intelligence, AAAI 2019, The Thirty-First Innovative Applications of Artificial Intelligence Conference, IAAI 2019, The Ninth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2019, Honolulu, Hawaii, USA, January 27 - February 1, 2019*, pp. 1352–1359. AAAI Press, 2019. doi: 10.1609/aaai.v33i01.33011352. URL <https://doi.org/10.1609/aaai.v33i01.33011352>.
- Amanda Rios and Laurent Itti. Closed-loop memory GAN for continual learning. In Sarit Kraus (ed.), *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI 2019, Macao, China, August 10-16, 2019*, pp. 3332–3338. ijcai.org, 2019. doi: 10.24963/ijcai.2019/462. URL <https://doi.org/10.24963/ijcai.2019/462>.
- Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022*, pp. 10674–10685. IEEE, 2022. doi: 10.1109/CVPR52688.2022.01042. URL <https://doi.org/10.1109/CVPR52688.2022.01042>.

- Axel Sauer, Katja Schwarz, and Andreas Geiger. Stylegan-xl: Scaling stylegan to large diverse datasets. In Munkhtsetseg Nandigjav, Niloy J. Mitra, and Aaron Hertzmann (eds.), *SIGGRAPH '22: Special Interest Group on Computer Graphics and Interactive Techniques Conference, Vancouver, BC, Canada, August 7 - 11, 2022*, pp. 49:1–49:10. ACM, 2022. doi: 10.1145/3528233.3530738. URL <https://doi.org/10.1145/3528233.3530738>.
- Thomas Schlegl, Philipp Seeböck, Sebastian M. Waldstein, Ursula Schmidt-Erfurth, and Georg Langs. Unsupervised anomaly detection with generative adversarial networks to guide marker discovery. In Marc Niethammer, Martin Styner, Stephen R. Aylward, Hongtu Zhu, Ipek Oguz, Pew-Thian Yap, and Dinggang Shen (eds.), *Information Processing in Medical Imaging - 25th International Conference, IPMI 2017, Boone, NC, USA, June 25-30, 2017, Proceedings*, volume 10265 of *Lecture Notes in Computer Science*, pp. 146–157. Springer, 2017. doi: 10.1007/978-3-319-59050-9_12. URL https://doi.org/10.1007/978-3-319-59050-9_12.
- Ari Seff, Alex Beatson, Daniel Suo, and Han Liu. Continual learning in generative adversarial nets. *CoRR*, abs/1705.08395, 2017. URL <http://arxiv.org/abs/1705.08395>.
- Gehui Shen, Song Zhang, Xiang Chen, and Zhi-Hong Deng. Generative feature replay with orthogonal weight modification for continual learning. In *International Joint Conference on Neural Networks, IJCNN 2021, Shenzhen, China, July 18-22, 2021*, pp. 1–8. IEEE, 2021. doi: 10.1109/IJCNN52387.2021.9534437. URL <https://doi.org/10.1109/IJCNN52387.2021.9534437>.
- Hanul Shin, Jung Kwon Lee, Jaehong Kim, and Jiwon Kim. Continual learning with deep generative replay. In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (eds.), *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pp. 2990–2999, 2017. URL <https://proceedings.neurips.cc/paper/2017/hash/0efbe98067c6c73dba1250d2beaa81f9-Abstract.html>.
- Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*, pp. 2818–2826. IEEE Computer Society, 2016. doi: 10.1109/CVPR.2016.308. URL <https://doi.org/10.1109/CVPR.2016.308>.
- Gido M Van de Ven, Hava T Siegelmann, and Andreas S Tolias. Brain-inspired replay for continual learning with artificial neural networks. *Nature communications*, 11(1):4069, 2020.
- Sakshi Varshney, Vinay Kumar Verma, P. K. Srijith, Lawrence Carin, and Piyush Rai. CAM-GAN: continual adaptation modules for generative adversarial networks. In Marc’Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan (eds.), *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pp. 15175–15187, 2021. URL <https://proceedings.neurips.cc/paper/2021/hash/8073bd4ed0fe0c330290c58056a2cd5e-Abstract.html>.
- Vinay Kumar Verma, Kevin J. Liang, Nikhil Mehta, Piyush Rai, and Lawrence Carin. Efficient feature transformations for discriminative and generative continual learning. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2021, virtual, June 19-25, 2021*, pp. 13865–13875. Computer Vision Foundation / IEEE, 2021. doi: 10.1109/CVPR46437.2021.01365. URL https://openaccess.thecvf.com/content/CVPR2021/html/Verma_Efficient_Feature_Transformations_for_Discriminative_and_Generative_Continual_Learning_CVPR_2021_paper.html.
- Chenshen Wu, Luis Herranz, Xialei Liu, Yaxing Wang, Joost van de Weijer, and Bogdan Raducanu. Memory replay gans: learning to generate images from new categories without forgetting. *CoRR*, abs/1809.02058, 2018. URL <http://arxiv.org/abs/1809.02058>.
- Weihao Xia, Yulun Zhang, Yujiu Yang, Jing-Hao Xue, Bolei Zhou, and Ming-Hsuan Yang. GAN inversion: A survey. *IEEE Trans. Pattern Anal. Mach. Intell.*, 45(3):3121–3138, 2023. doi: 10.1109/TPAMI.2022.3181070. URL <https://doi.org/10.1109/TPAMI.2022.3181070>.
- Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *CoRR*, abs/1708.07747, 2017. URL <http://arxiv.org/abs/1708.07747>.
- Xin Yi, Ekta Walia, and Paul S. Babyn. Generative adversarial network in medical imaging: A review. *CoRR*, abs/1809.07294, 2018. URL <http://arxiv.org/abs/1809.07294>.

- Mengyao Zhai, Lei Chen, Frederick Tung, Jiawei He, Megha Nawhal, and Greg Mori. Lifelong GAN: continual learning for conditional image generation. In *2019 IEEE/CVF International Conference on Computer Vision, ICCV 2019, Seoul, Korea (South), October 27 - November 2, 2019*, pp. 2759–2768. IEEE, 2019. doi: 10.1109/ICCV.2019.00285. URL <https://doi.org/10.1109/ICCV.2019.00285>.
- Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018*, pp. 586–595. Computer Vision Foundation / IEEE Computer Society, 2018. doi: 10.1109/CVPR.2018.00068. URL http://openaccess.thecvf.com/content_cvpr_2018/html/Zhang_The_Unreasonable_Effectiveness_CVPR_2018_paper.html.
- Jun-Yan Zhu, Taesung Park, Phillip Isola, and Alexei A. Efros. Unpaired image-to-image translation using cycle-consistent adversarial networks. In *IEEE International Conference on Computer Vision, ICCV 2017, Venice, Italy, October 22-29, 2017*, pp. 2242–2251. IEEE Computer Society, 2017. doi: 10.1109/ICCV.2017.244. URL <https://doi.org/10.1109/ICCV.2017.244>.

A APPENDIX

In this appendix, we describe the implementation details and the values of hyperparameters. Also, we present additional experimental results, including (1) the GCL performance comparison using SVHN, and (2) more qualitative examples in CIFAR-10. Our code is available at the following anonymous website: <https://github.com/bigdata-inha/RactoFit>.

A.1 IMPLEMENTATION DETAILS

Training on MNIST and FMNIST. We set the dimensionality of latent vectors (i.e., $\mathbf{z}$) to 56, and set the batch size to 64. For each continual task, we train the GAN model for 50 epochs, employing Adam optimizer with $\beta_1 = 0.0$, $\beta_2 = 0.9$ and a learning rate of 0.002.

Training on CIFAR-10 and SVHN. With 128-dimensional latent vectors and the batch size of 256, we incrementally train the GAN for 1000 epochs on each continual task, using a 5 : 1 ratio for the discriminator and generator on the number of training iterations. We also use the same Adam optimizer with $\beta_1 = 0.0$, $\beta_2 = 0.9$ and the learning rate of 0.002 as in MNIST and FMNIST.

Training and Measurement Details. For each replay-based method, we replay the same number of synthetic images for each previous task, which is commonly set to 5000 in all datasets as it is the exact (CIFAR-10) or approximate (MNIST, FMNIST, and SVHN) number of real images in each single task. To train with CloGAN (Ríos & Itti, 2019), we basically follow the same settings as in the original version. For EWC (Kirkpatrick et al., 2016), we set λ to 70 in MNIST and FMNIST, and 500 in CIFAR-10 and SVHN. In our RactoFit method, we construct the set of synthetic images for each previous task in a way that the total number of images per task is still be the same (i.e., 5000), including all the images generated from either buffered or optimized vectors. When we combine MeRGAN (Wu et al., 2018) or DGR (Shin et al., 2017) with Rehearsal, we give them even a favorable condition that the total number of synthetic images is fixed to 5000 regardless of the number of real images stored in the buffer. In the inversion loss function of Eq. (5), we adopt L2 norm with $\beta = 0$ for MNIST and FMNIST, and L1 norm with $\beta = 10$ for SVHN and CIFAR-10. In order to measure fitting capacity or FID based on the expert model, we use Adam optimizer with a learning rate of 0.01 and train each corresponding model for 50 epochs in MNIST, FMNIST, and SVHN, while employing an existing pretrained ResNet-18 for the CIFAR-10 dataset. In all datasets, we generate 5000 synthetic images for each task and measure their FID score and fitting capacity.

Memory Complexity. For a fair comparison in experiments, one might consider including the storage requirements of the previous model. However, the storage for the previous model should not be equated with the memory needed for images, as it is only required during the training phase. When a new task arrives, we create a copy of the current model, and this copy is discarded after training. In contrast, the memory for images and latent vectors must be maintained permanently. Therefore, our experiments focus on permanent memory rather than working memory.

A.2 ADDITIONAL EXPERIMENTAL RESULTS

A.2.1 RESULTS WITH SVHN

Table A1: GCL performance comparison upon the completion of all continual tasks on SVHN.

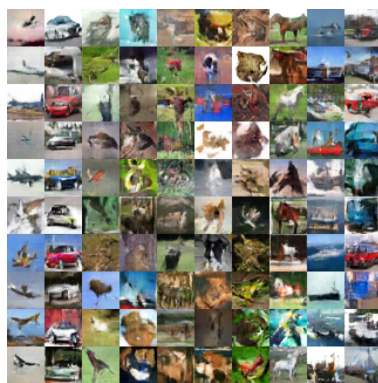
SVHN			
Methods	Buffer (%) ($ \mathcal{M} : \mathcal{Z} $)	FID (expert)	fitting capacity
Upperbound	100%	0.29	87.47
EWC	0%	10.70	11.57
MeRGAN		0.77	29.98
RactoFit (ours)		0.63	43.64
Rehearsal	0.8% (0.5K : 12K)	2.16	48.45
MeRGAN + Rehearsal		0.63	43.35
CloGAN		2.81	30.60
RactoFit (ours)		0.61	51.69
Rehearsal	1.6% (1K : 24K)	1.30	62.80
MeRGAN + Rehearsal		0.69	64.51
CloGAN		2.80	34.65
RactoFit (ours)		0.45	69.80

SVHN consists of 73,257 training images along with 26,032 test images distributed across 10 classes. We use cropped images centered around a single character, which are of 32×32 size. Similar to the other datasets, we construct a sequence of 10 tasks, each corresponding to a single class. All details, including model architecture, training process, and the size of the memory buffer, are the same as those used for CIFAR-10.

Table A1 shows that our RactoFit method manages to achieve the best GCL performance in SVHN as well, while confirming its effective memory utilization based on the comparison to Rehearsal with the same memory space.

A.2.2 MORE QUALITATIVE EXAMPLES

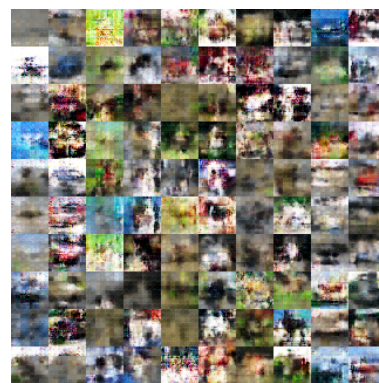
In Figure A1, we present more generated images from each method after the completion of training all 10 tasks of CIFAR-



(a) Upperbound



(b) EWC



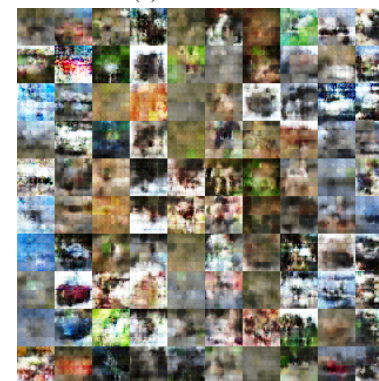
(c) MeRGAN



(d) RactoFit (0%)



(e) Rehearsal (0.8%)



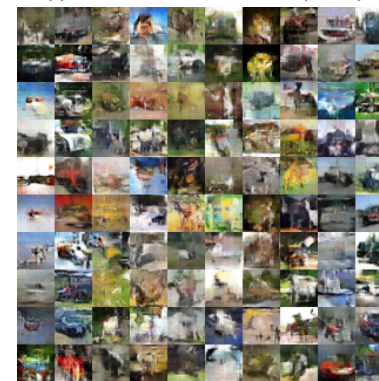
(f) MeRGAN + Rehearsal (0.8%)



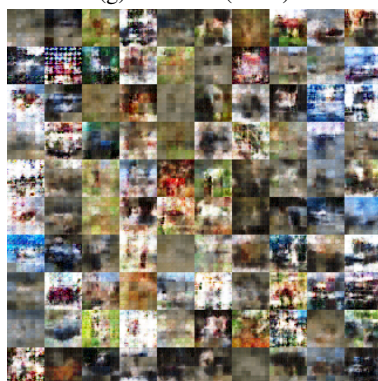
(g) CloGAN (0.8%)



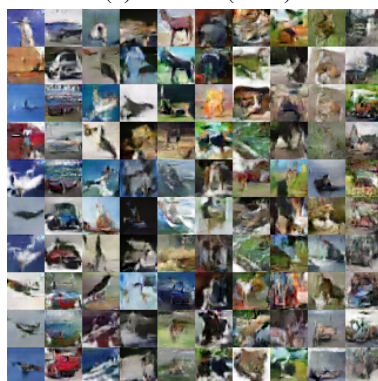
(h) RactoFit (0.8%)



(i) Rehearsal (1.6%)



(j) MeRGAN + Rehearsal (1.6%)



(k) CloGAN (1.6%)



(l) RactoFit (1.6%)

Figure A1: Generated samples from each method after continual learning of all tasks on CIFAR-10.

10, including the ones in Figure 5. In each method, the columns correspond to 10 classes in CIFAR-10, which are ‘plane’, ‘car’, ‘bird’, ‘cat’, ‘deer’, ‘dog’, ‘frog’, ‘horse’, ‘ship’, and ‘truck’.